Using **Inspect Element in Chrome** is very useful for a DevOps engineer—especially for debugging frontend issues, checking API calls, verifying deployments, and troubleshooting production problems.

Below is a practical, DevOps-focused guide.

🔹 How to Open Inspect Element in Chrome

Method 1 (Mouse):

- Right-click on any webpage → **Inspect**

Method 2 (Keyboard shortcuts):

- Windows/Linux: Ctrl + Shift + I
- Mac: Cmd + Option + I

Direct DevTools:

- Press F12

🔹 Key Tabs DevOps Engineers Actually Use

1️⃣ Elements Tab (UI Debugging)

Purpose: Inspect HTML & CSS

DevOps use cases:

- Verify if latest UI is deployed
- Check if CDN is serving correct files
- Debug broken layouts after deployment
- Identify missing static assets

What to look for:

- HTML structure
- CSS styles
- class/id names
- image paths

✅ Example: After deployment, button style looks wrong → check CSS loaded or not.

2️⃣ Network Tab (🔥 MOST IMPORTANT for DevOps)

This is your **primary weapon**.

Purpose: Monitor all network requests.

DevOps use cases:

- Check API failures (500, 404, 403)
- Verify backend connectivity
- Debug CORS issues
- Confirm load balancer routing
- Check caching problems
- Verify environment variables in frontend builds

✅ How to Use Network Tab

1. Open DevTools → **Network**
2. Refresh page (Ctrl + R)
3. Filter by:
    - **Fetch/XHR** → APIs
    - **JS** → scripts
    - **CSS**
    - **Img**

✅ What Status Codes Mean

Code	Meaning	DevOps Action
200	Success	OK
304	Cached	Check cache rules
404	Not Found	Check routing/CDN
403	Forbidden	Check IAM/security
500	Server error	Check backend logs
502	Bad gateway	Check Nginx/ALB
504	Timeout	Check upstream service

3️⃣ Console Tab (Runtime Errors)

Purpose: View JavaScript errors and logs.

DevOps use cases:

- Detect frontend crashes
- Identify missing environment variables
- Debug production JS errors
- Check CSP violations

✅ Common errors:

- Failed to fetch
- CORS policy blocked
- Uncaught TypeError

4️⃣ Application Tab (Very Important in Production)

Purpose: Inspect browser storage.

DevOps use cases:

- Check JWT tokens
- Verify cookies
- Debug session issues
- Inspect localStorage

Sections to check:

- Cookies
- Local Storage
- Session Storage

5️⃣ Sources Tab (Advanced Debugging)

Purpose: View loaded JS files.

DevOps use cases:

- Verify correct build version
- Check source maps
- Debug production bundles

🔥 Real DevOps Scenarios

✅ Scenario 1: API Not Working After Deployment

Steps:

1. Open Network → Fetch/XHR
2. Refresh page
3. Click failed API
4. Check:
    - Status code
    - Request URL
    - Response body

👉 Then check:

- backend logs
- Kubernetes pod logs
- Nginx logs
- ALB target health

✅ Scenario 2: CORS Issue

In Console you'll see:

Access to fetch at ... blocked by CORS policy

DevOps fix:

- Update backend CORS headers
- Check API Gateway config
- Verify Nginx headers

✅ Scenario 3: Old UI Showing After Deployment

Check in Network:

- JS files cached?
- Response header: cache-control
- CDN caching?

Fix:

- Invalidate CDN
- Change build hash
- Update cache headers

✅ Scenario 4: 502 Bad Gateway

Check Network → failing request.

Then verify:

- Nginx upstream
- Kubernetes service
- Pod health
- Load balancer target group

🔹 Pro Tips for DevOps Engineers

✅ Always **hard refresh** after deployment
Ctrl + Shift + R

✅ Disable cache while debugging
Network tab → ✓ Disable cache

✅ Use filter:

-status-code:200

to find failures quickly.

✅ Preserve logs
Network → ✓ Preserve log

🚀 DevOps Power Workflow (Recommended)

When something breaks in production:

1. Open Inspect
2. Go to Network
3. Refresh page
4. Find red requests
5. Check response
6. Correlate with server logs
7. Fix infra/backend
8. Redeploy
9. Verify again

If you want, I can next show:

- 🔥 Inspect Element for Kubernetes apps
- 🔥 How to debug Nginx using Chrome
- 🔥 How to trace API → Load Balancer → Pod flow

Just say the word 👍